

* OOPS :- Object Oriented Programming is a programming set of concepts that organizes software design around objects rather than functions and logic.

- The Core Idea → Instead of just writing a list of instructions, we represent real-world entities as Objects. These objects interact with each other, much like they do in real life.

- Object Anatomy → Every object consists of two main components:

1. Characteristics : Unique identifiers (e.g. a Car has a model & Brand).

2. Behaviours : Actions the object can perform (e.g. Start, Stop, Shift Gear).

- Classes as Blueprints → A class acts as a template or blueprint. You define the class once, and then you can create as many specific Objects (instances) of that class as needed.

- Pillars of OOPS :-

1. Abstraction
2. Encapsulation
3. Inheritance
4. Polymorphism

* Abstraction :-

It is the practice of hiding unnecessary implementation details from the user while exposing only the essential interface needed to interact with an object.

- Real-World Analogy → When driving a car, you use the steering wheel, pedals, and gear shifter. You do not need to understand the complex internal mechanics of the engine or transmission to drive effectively.

- The Goal → Abstraction simplifies the complexity for the clients (the users) and reduces cognitive load by showing only what is necessary.

- Code Representation →

→ We use abstract classes and interfaces. This defines what a car can do (e.g. `startEngine()`, `accelerate()`) without defining how it does it internally.

→ The actual logic is implemented in a child class (like a `SportsCar`), which keeps the implementation details hidden from the end user.

* Encapsulation :-

It focuses on building data (characteristics) and methods (behaviours) into a single unit (a class) and controlling access to that data to ensure security.

- The Capsule Concept → Think of a medicine capsule - it protects the contents inside. Similarly, a class acts as a protective container for its data.
- Data Security & Access Modifiers → Unlike abstraction (which hides complexity), encapsulation hides data for security.
 - Public :- Members are accessible from outside the class.
 - Private :- Members are restricted to the class itself, preventing external code from corrupting sensitive data.
- Getters & Setters → Since private data cannot be accessed directly, we provide public "getter" and "setter" methods. This allows us to perform validation before modifying or retrieving data, ensuring the object stays in a valid state.

* Inheritance :-

It represents the 'parent-child' relationship between objects in real life and programming. It allows a 'child' class to acquire the properties and behaviour of a 'parent' class, promoting code reusability.

• Concepts & Analogy →

Parent Class ⇒ (Base Class) Represents generic attributes (e.g. general Car with Brand, Model).

Child class ⇒ (Derived Class) Represents specialized versions (e.g. ManualCar or ElectricCar).

These classes inherit everything from parent but add their own unique features.

• Access Modifiers →

Public ⇒ Inherited public members stay public, and protected stay protected.

Private/Protected ⇒ Using these instead of public restricts how future classes can inherit from the child class.

★ In 99% of ILO scenarios, public inheritance is used to maintain core principle of OOP.

• Key Benefits →

→ Avoids rewriting code for common properties (e.g. Model, Brand).

→ Simplifies maintenance of complex systems by organizing objects into hierarchical structures.

* Polymorphism :-

It means 'many forms' and allows objects to respond to same command in different ways depending on their type or specific situation.

→ Dynamic Polymorphism (Method Overriding)

- Concept → Different classes (e.g. Manual vs. Electric) share a common base action (like Accelerate), but each implements it uniquely.

- Mechanism → Uses abstract functions in base class. The specific function is determined at runtime based on the object type.

- Example → Both cars have an Accelerate function, but the ElectricCar manages battery consumption while ManualCar handles gears.

→ Static Polymorphism (Method Overloading)

- Concept → A single class has multiple versions of same function name, differing by input parameters.

Date.....

- Mechanism → Resolved at compile-time. It selects the appropriate method based on argument types passed at call site. Return type alone cannot be used to differentiate overloaded methods.
- Example → Two Accelerate functions in a class - one takes no arguments (default speed), and the other one takes a 'speed' argument for custom acceleration.

→ Key Takeaway

- Overriding → Changing behaviour across class hierarchies.
- Overloading → Changing behaviour within the same class using different signatures.